

Project: Semester Project: Smart Parking Platform

Version: 3.0

Date: 09.017.2026

1. Introduction

WE ARE \$oftware Corp. A fictitious software development studio with unlimited resources and budget. The project involves the planning and management of a software system that helps drivers locate, reserve, and pay for parking spaces for parking garages in real time. The platform will be available through web and mobile applications and will provide parking availability information, navigation assistance, and management tools for parking operators. The goal is to develop a project management plan for the creation of this system, including requirements gathering, stakeholder management, scheduling, budgeting, risk management, and team coordination.

Aiming to do more than our competitors:

[Parknav® - Find Real Time On-Street and Off-Street Parking](#)

Just shows parking, cant pay for it ahead of time

[SpotHero | Get Everywhere Easier | Reserve Parking and Save](#)

Doesnt show a real time GPS or open parking, you need to know the address before hand

[Spot – Your Smart Parking Companion](#)

Has a waitlist to even join the app and not usable on non-mobile devices

[Download SpotHero - Find Parking for Android & iOS - Mobexer](#)

Known to only work in small towns with inaccurate pricing info, crashes unexpectedly. And slow support

1.1 Purpose

This document defines the requirements for Semester Project: Smart Parking Platform.

1.2 Scope

Drivers often spend significant time searching for available parking, leading to congestion, frustration, and lost productivity. Parking operators need better tools to monitor occupancy, manage pricing, and improve utilization of parking assets.

Hopes to do all the competition can do and more

Out of scope would be things that require more time such as:

Communication with other customers through the app, making it a pseudo social media platform.

It's own GPS system requiring dedicated owned satellites without connecting to others

Extra addons without proper time slots

Calling through the app without using the dedicated methods such as the customers' normal phone plan.

2. Overall Description

- Users: <List the different groups of users that will be using the finished product>
- System Environment: Web-based / desktop / mobile application
- Constraints: Must run on specific hardware, must require specific authentication, etc.
- Assumptions: Application has internet / intranet access, etc.

3. Functional Requirements

The platform should provide:

User registration and authentication

Real-time parking space availability
Interactive map displaying available parking locations
Parking reservation capability
Digital payment processing
Reservation history and receipts
Administrative dashboard for parking operators
Occupancy reporting and analytics
Notifications and alerts for users
Integration with external mapping/navigation services

4. Non-Functional Requirements

NFR1: The system shall support at least 1,000 concurrent users.
NFR2: The system shall update parking values within 1 second.
NFR3: The system shall provide 99.99% uptime.
NFR4: The system shall provide a visual parking gps/map
NFR5: The system shall provide support lines for the app or dedicated parking area
NFR6: The system shall be able to save info such as insurance info.
NFR7: The system shall be able to save info such as roadside assistance info.

5. Use Case Example

Case 1:

Title: Nearest parking search (GPS map)

Actor: Customer

Precondition: User is logged in, using search query in GPS

Steps:

1. User selects filters out the kind of parking they want
2. User searches for said parking
3. User views available parking
4. User chooses the parking

Postcondition: user has selected the parking they desire and can go to it or pay for it

Case 2:

Title: Nearest parking search (Catalogue grid)

Actor: Customer

Precondition: User is logged in, using search query in catalogue

Steps:

1. User selects filters out the kind of parking they want
2. User searches for said parking
3. User views available parking
4. User chooses the parking

Postcondition: user has selected the parking they desire and can go to it or pay for it

Case 3:

Title: creating an account

Actor: Customer

Precondition: User downloads the app or uses the website

Steps:

1. User clicks "Sign up"
2. User adds their email
3. User verifies their email
4. User creates their username and password
5. User add two factor authentication (optional)

6. User adds payment info during account creation to be saved to said account (optional)

7. User inserts and saved driver information to their account

Postcondition: user has created an account

Case 4:

Title: Adding/saves Payment info (internal, before parking)

Actor: Customer

Precondition: User is logged in, has payment info to save

Steps:

1. User selects the setting menu

2. User selects payment options

3. User puts in payment info with secure vendor/ or uses saved data on device

4. User can use said info during parking

Postcondition: user has saved a payment option to their account

Case 5:

Title: Adding/saves Payment info (internal, during parking)

Actor: Customer

Precondition: User is logged in, user has the parking they want to pay for in view

Steps:

1. User selects the parking

2. User selects payment options

3. User puts in payment info with secure vendor/ or uses saved data on device

4. User pays for parking

Postcondition: user has saved a payment option to their account

Case 6:

Title: Adding/saves insurance for ease of access (optional)

Actor: Customer

Precondition: User is logged in

Steps:

1. User selects the account
2. User selects insurance info options
3. User puts in insurance info with a printable file format
4. User can view their insurance by going to their profile (local)

Postcondition: user has saved their insurance files locally for ease of access

Case 7:

Title: Viewing parking history

Actor: Customer

Precondition: User is logged in

Steps:

1. User selects the account
2. User selects "Parking History"
3. User selects past parking from a list

Postcondition: user will be able to view parking time, date, and payment history

Case 8:

Title: Reserving parking

Actor: Customer

Precondition: User is logged in, user has the parking they want to in view

Steps:

1. User selects the parking
2. User selects payment options (optional based on vendor)

3. User chooses to reserve the spot based on the vendor

Postcondition: user has successfully reserved the parking (if allowed by the vendor)

Case 9:

Title: Locating route to parking Via GPS

Actor: Customer

Precondition: User is logged in, user has the parking they want in view

Steps:

1. User selects the parking

2. User selects chooses routes based on filters such as tolls and time

3. User can proceed to the parking via the route

Postcondition: user has been routed to the chosen parking

Case 10:

Title: Add new vendors

Actor: network admin, Cloud service

Precondition: Vendor is locatable via the web/ database

Steps:

1. User selects available vendors

2. User scans new vendor based on any database they may have

3. User enters the vendor into the application via a dedicated dictionary/table

4. User saves the new vendor with parking being updated for all users

Postcondition: Mor parking/ vendors should show up in the application/ database

Case 11:

Title: Remove vendors

Actor: network admin, Cloud service

Precondition: Vendor is locatable via the web/ database

Steps:

1. User selects desired vendors from the database
2. User removes said vendors

Postcondition: the database should update removing the vendor and their parking

Case 12:

Title: Display unaffiliated Vendors

Actor: network admin, Cloud service

Precondition: Vendor is locatable via the web/ database

Steps:

1. User selects available vendors database
2. User scans for available non associated parking vendors
3. User chooses to toggle on the ability to use said vendors for the application

Postcondition: More parking/ vendors should show up as unaffiliated in the application/ database (optional, client-side toggle for customers as well)

Case 13:

Title: Display unaffiliated Vendors

Actor: Customer

Precondition: user is logged in

Steps:

1. User selects the setting menu
2. User scans for available non associated parking vendors
3. User chooses to toggle on the ability to use said vendors for the application

Postcondition: More parking/ vendors should show up as unaffiliated in the application/ database

Case 14:

Title: Save local cache and map

Actor: Customer

Precondition: user is logged in has loaded a local map of available parking before

Steps:

1. User selects the setting menu
2. User scans for available non associated parking vendors
3. User saves a snapshot of local parking and their prices

Postcondition: Parking should be saved locally in case the user loses access to internet. (note, values will not update)

Case 15:

Title: Call for roadside assistance

Actor: Customer

Precondition: user is logged, user has saved roadside assistance info to their account

Steps:

1. User selects their profile
2. User selects the insurance tab
3. User selects the roadside assistance tab
4. User can call for roadside assistance/ view their number (mobile only)
5. User can send roadside assistance their location via text (mobile only)

Postcondition: Roadside assistance will be notified

Case 16:

Title: Account deletion

Actor: Customer

Precondition: User is logged in

Steps:

1. User selects their profile
2. User selects their profile settings
3. User selects delete account
4. User confirms account deletion after re logging the password and verifying through email

Postcondition: The users account will be deleted.

Part 2 down:

1. Frontend

1.1 Account Management

- Sign Up
- Email verification
- Username/password creation
- 2FA (optional)
- Account Deletion
- Password confirmation
- Email confirmation

1.2 Parking Search

- GPS Map Search
- Parking filters
- Available parking display
- Catalogue Grid Search
- Parking filters
- Grid view display

1.3 Parking Selection

- Parking details
- Parking availability
- Vendor information

1.4 Navigation

- GPS routing
- Route generation
- Toll avoidance
- Travel time filters

1.5 User Profile

- Driver information
- Insurance storage
- Parking history
- Roadside assistance

1.6 Settings

- Payment management
- Unaffiliated vendor toggle
- Offline map cache

2. Payment System

2.1 Payment Methods

- Add payment information
- Edit payment information
- Remove payment information

2.2 Parking Transactions

- Pay for parking
- Save payment during checkout
- Payment confirmation

2.3 Reservation Processing

- Vendor reservation support
- Reservation confirmation

3. Backend Services

3.1 Authentication

- Login
- Email verification
- Password management
- Two-factor authentication

3.2 User Data Management

- Customer profiles
- Driver records
- Insurance documents
- Parking history records

3.3 Parking Database

- Parking inventory
- Vendor records
- Parking availability

3.4 Map Services

- GPS integration
- Route calculation
- Offline cache synchronization

4. Vendor Management

4.1 Vendor Administration

- Add vendors
- Update vendor data
- Remove vendors

4.2 Vendor Discovery

- Scan for new vendors
- Database synchronization

4.3 Unaffiliated Vendors

- Detect unaffiliated vendors
- Enable vendor visibility

- Customer/vendor toggle controls

5. Cloud & External Integrations

5.1 Payment Gateway

- Secure payment processing
- Transaction validation

5.2 Mapping Services

- GPS service integration
- Route generation

5.3 Insurance Services

- Insurance file storage
- Roadside assistance data

5.4 Notification Services

- Email verification
- Account deletion confirmation
- Reservation confirmations

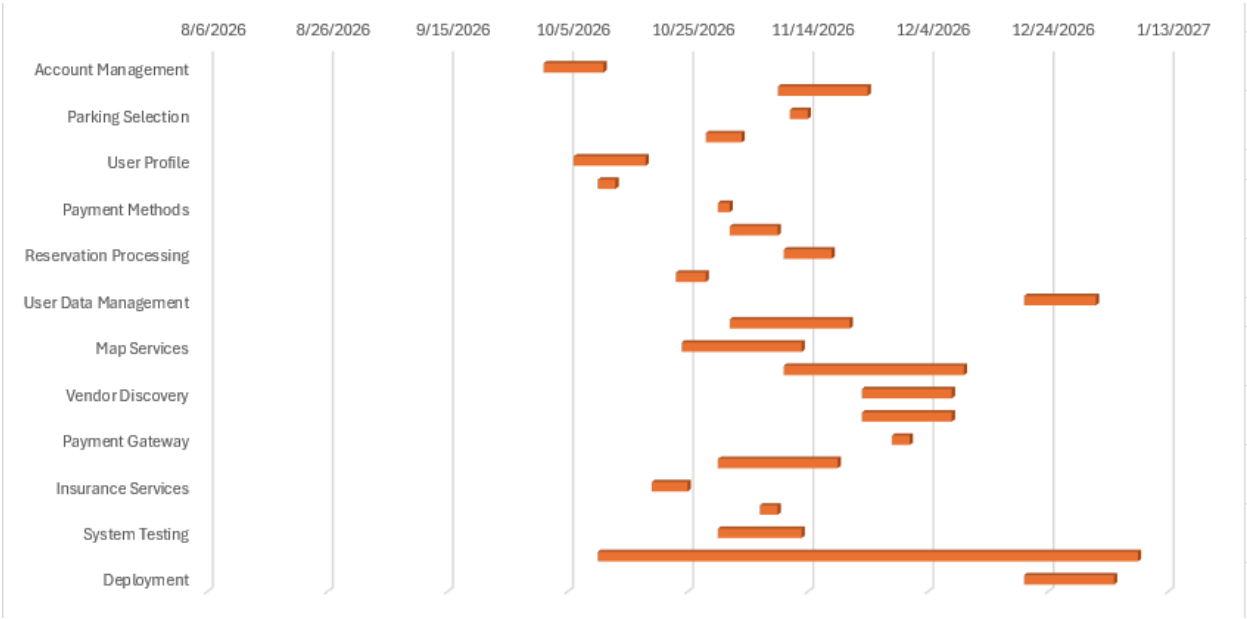
Market rates: Estimated total cost: 120,000\$ - 200,000\$ (budget buffer included)

[Auto GPS tracker app: Cost for bespoke development](#)

[How to Create an App and Make Money in 2026 \(Complete Guide\)](#)

[Smart Parking App Development Cost Breakdown](#)

Gant Chart timeline (based on WBS):



Trello Board:

Trello

Search

Create

% 14 days left

RD

Atlassian uses cookies to improve your browsing experience, perform analytics and research, and conduct advertising. Accept all cookies to indicate that you agree to our use of cookies on your device. [Atlassian cookies and tracking notice](#)

PreferencesOnly necessaryAccept all

CIS 4374

RD

Share

Login15

Create user registration page

Validate email type

Send email verification

Verify verification

Create username and password

Login with username and password

Password Reset

Password reset confirmation

Optional two factor authentication

+ Add a card

UI for operator and user17

GPS- map view

GPS parking view

Display available parking on GPS

Display unavailable parking on GPS

Filter parking buy price

filter parking by distance

Display parking grid options

View parking details

Show vendor information

+ Add a card

Backend processes for operator and user20

User authentication

email verification

Password management

Two factor authentication

Customer profile storage

Driver information storage

Insurance document storage

Parking history database

Parking inventory

+ Add a card

+ Add another list

Inbox

Planner

Board

Switch boards